

STABILITY

- API Stability Policy
 - Semantic versioning
 - Public API surface
 - Deprecation flow
 - What does not count as a break
 - What does count as a break
 - C++ ABI
 - How to file a stability complaint

API Stability Policy

This document defines what counts as a breaking change for agentcore, what the deprecation flow looks like, and what callers can rely on.

Semantic versioning

We follow [SemVer 2.0.0](#) once we reach **1.0**. Until then (0.x.y):

- **0.x.0 → 0.(x+1).0** can break public API. Read the CHANGELOG.
- **0.x.y → 0.x.(y+1)** must be backwards-compatible: bug fixes, internal refactors, new opt-in features only.

Public API surface

A symbol is **public** if it is exported from agentcore.__all__ or its containing submodule's __all__. Public symbols belong to one of three tiers, labeled in the docstring:

Tier	Stability promise
# api: stable	Will not break in a minor release without a one-release-cycle deprecation period.
# api: experimental	May change between minor releases. Always opt-in.
# api: internal	Will change without notice. Do not depend on this from external code. Names starting with _ are also internal.

If a symbol has no marker, treat it as **experimental** until we add one.

v0.1 stable surface

The following symbols are guaranteed stable at v0.1.0:

- agentcore.Agent
- agentcore.Runtime
- agentcore.Message, agentcore.Role
- agentcore.MockProvider, agentcore.PyProviderBase
- agentcore.GenerationRequest, agentcore.GenerationResponse
- agentcore.CancelToken, agentcore.OverflowPolicy
- agentcore.tool, agentcore.ToolBox
- agentcore.Graph, agentcore.GraphResult, agentcore.GraphExhausted, agentcore.run_graph

- `agentcore.AsyncRuntime`, `agentcore.AsyncAgent`, `agentcore.to_thread`
- `agentcore.StateStore`, `agentcore.InMemoryStateStore`, `agentcore.SQLiteStateStore`
- `agentcore.RateLimiter`, `agentcore.RetryPolicy`
- `agentcore.TraceSink`, `agentcore.NullTraceSink`, `agentcore.PrintTraceSink`, `agentcore.OpenTelemetryTraceSink`
- `agentcore.UsageTracker`, `agentcore.UsageRecord`

v0.1 experimental surface

These work today but may change shape before 1.0:

- `agentcore.providers.*` — provider implementations (OpenAI/Anthropic/Ollama)
- The exact text of error messages (do not match on them)
- The structure of `agentcore.tools._default_error_redactor` output

Internal

- `agentcore._agentcore.*` — the raw Pybind11 module. Use the Python wrappers.
- Anything not listed under stable or experimental.

Deprecation flow

Stable symbols are removed across at least one minor version:

1. Release **N**: deprecate the symbol. It still works. A `DeprecationWarning` is raised on first use. `CHANGELOG.md` lists the deprecation.
2. Release **N+1**: deprecation persists. Documentation marks the symbol as deprecated and points to the replacement.
3. Release **N+2** (earliest): symbol removed.

For 0.x development, "minor" means "the next 0.x.0 release."

What does not count as a break

- Adding new public symbols.
- Adding new optional parameters to functions (with backwards-compatible defaults).
- Adding new fields to dataclasses with default values.
- Changing the wording of log messages.
- Changing internal implementation that does not affect behavior.
- Tightening type hints (e.g. `Optional[X] → X` when `None` was never valid).
- Bug fixes — even ones that change observable behavior, when the previous behavior was clearly incorrect.

What does count as a break

- Removing a public symbol without going through deprecation.
- Adding a required parameter.
- Changing the type of a public field, return value, or argument.
- Reordering positional parameters.
- Changing the meaning of an enum value.
- Tightening preconditions (e.g. now raising for an input that previously worked).
- Loosening postconditions in a way callers can observe.

C++ ABI

The C++ static library `libagentcore_core.a` does **not** have ABI stability yet. The C++ headers are not versioned; downstream C++ consumers should pin to a specific git commit and rebuild on upgrade. C++ ABI stability is a v1.0 goal, not a v0.1 promise.

How to file a stability complaint

If we break something we shouldn't have, file an issue with the regression label and a minimal repro. We will treat unintentional breakage of a stable symbol as a release-blocking bug.